Understanding the Core Concepts of Laravel 5

As the title of this chapter suggests, we will be providing a general overview of the Laravel framework, covering the main concepts related to the development of web applications using a web services architecture. More precisely, we will use a RESTful architecture in this book.

We assume that you already have a basic understanding of the RESTful architecture and how web services (here, we call them **Application Programming Interface** (**API**) endpoints) work.

However, if you are new in this concept, don't worry. We will help you get started.

The Laravel framework will be a helpful tool because with it, all of the data inside our controllers will be converted to the JSON format, by default.

The Laravel framework is a powerful tool for the development of web applications, using the paradigm *convention over configuration*. Out of the box, Laravel has all of the features that we need to build modern web applications, using the **Model View Controller** (**MVC**). Also, the Laravel framework is one of the most popular PHP frameworks for developing web applications today.

From now until the end of this book, we will refer to the Laravel framework simply as Laravel.

The Laravel ecosystem is absolutely incredible. Tools such as Homestead, Valet, Lumen, and Spark further enrich the experience of web software development using PHP.

There are many ways to start developing web applications using Laravel, meaning that there are many ways to configure your local environment or your production server. This chapter does not favor any specific way; we understand that each developer has his or her own preferences, acquired over time.

Regardless of your preferences for tools, servers, virtual machines, databases, and so on, we will focus on the main concepts, and we will not assume that a certain way is right or wrong. This first chapter is just to illustrate the main concepts and the actions that need to be performed.

Keep in mind that regardless of the methods you choose (using Homestead, WAMP, MAMP, or Docker), Laravel has some dependencies (or server requirements) that are extremely necessary for the development of web applications.

> *You can find more useful information in the official Laravel documentation at https://laravel.com/docs/5.6.*

In this chapter, we will cover the following points:

- Setting up the environment
- The basic architecture of a Laravel application
- The Laravel application life cycle
- Artisan CLI
- MVC and routes
- Connecting with the database

Setting up the environment

Remember, no matter how you have configured your environment to develop web applications with PHP and Laravel, keep the main server requirements in mind, and you will be able to follow the examples in this chapter.

It is important to note that some operating systems do not have PHP installed. As this is the case with Windows machines, here are some alternatives for you to create your development environment:

- HOMESTEAD (recommended by Laravel documentation): https://laravel.com/docs/5.6/homestead
- MAMP: https://www.mamp.info/en/
- XAMPP: https://www.apachefriends.org/index.html
- WAMP SERVER (only for Windows OS): http://www.wampserver.com/en/
- PHPDOCKER: https://www.docker.com/what-docker

Installing Composer package manager

Laravel uses **Composer**, a dependency manager for PHP, very similar to **Node Package Manager (NPM)** for Node.js projects, PIP for Python, and Bundler for Ruby. Let's see what the official documentation says about it:

"A Composer is a tool for dependency management in PHP. It allows you to declare the libraries your project depends on and it will manage (install/update) them for you."

So, let's install Composer, as follows:

Go to https://getcomposer.org/download/ and follow the instructions for your platform.

> *You can get more information at https://getcomposer.org/doc/00-intro.md.*

Note that you can install Composer on your machine locally or globally; don't worry about it right now. Choose what is easiest for you.

All PHP projects that use Composer have a file called composer.json at the root project, which looks similar to the following:

```
{
 "require": {
    "laravel/framework": "5.*.*",
 }
}
```

This is also very similar to the package.json file on Node.js and Angular applications, as we will see later in this book.

> *Here's a helpful link about the basic commands: https://getcomposer.org/doc/01-basic-usage.md*

Installing Docker

We will use Docker in this chapter. Even though the official documentation of Laravel suggests the use of Homestead with virtual machines and Vagrant, we chose to use Docker because it's fast and easy to start, and our main focus is on Laravel's core concepts.

You can find more information about Docker at https://www.docker.com/what-docker.

As the Docker documentation states:

Docker is the company driving the container movement and the only container platform provider to address every application across the hybrid cloud. Today's businesses are under pressure to digitally transform, but are constrained by existing applications and infrastructure while rationalizing an increasingly diverse portfolio of clouds, datacenters, and application architectures. Docker enables true independence between applications and infrastructure and developers and IT ops to unlock their potential and creates a model for better collaboration and innovation.

Let's install Docker, as follows:

1. Go to https://docs.docker.com/install/.
2. Choose your platform and follow the installation steps.
3. If you have any trouble, check the getting started link at https://docs.docker.com/get-started/.

As we are using Docker containers and images to start our application and won't get into how Docker works behind the scenes, here is a short list of some Docker commands:

Command:	**Description**:
docker ps	Show running containers
docker ps -a	Show all containers
docker start	Start a container
docker stop	Stop a container
docker-compose up -d	Start containers in background
docker-compose stop	Stop all containers on docker-compose.yml file
docker-compose start	Start all containers on docker-compose.yml file
docker-compose kill	Kill all containers on docker-compose.yml file

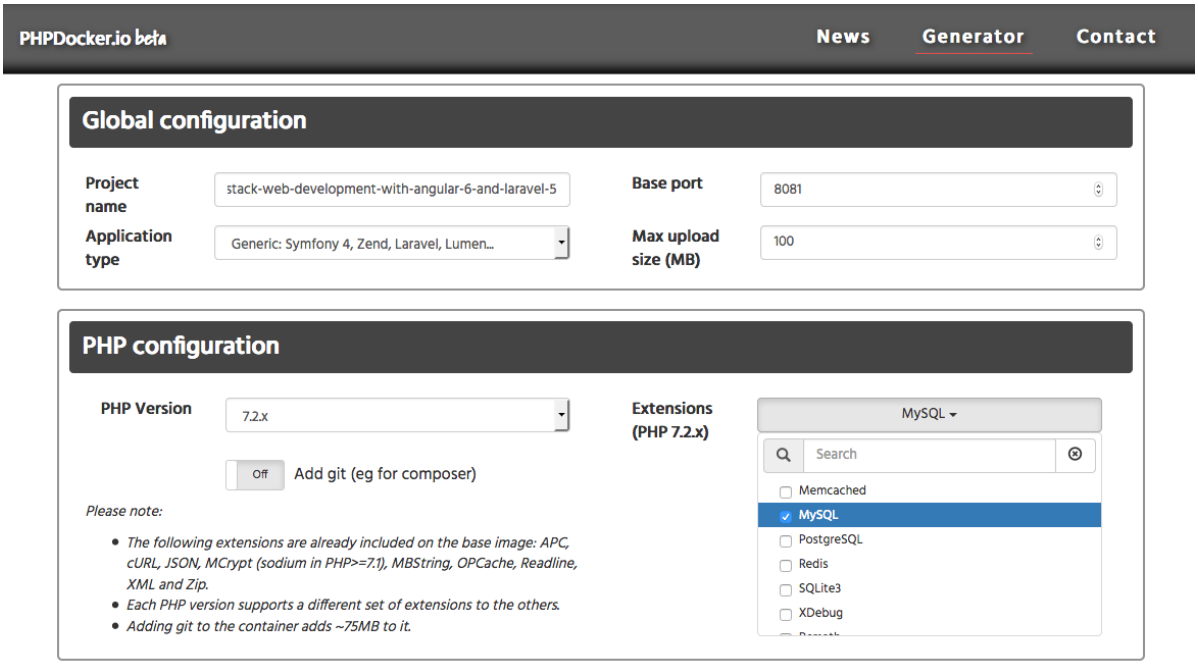
docker-compose logs	Log all containers on docker-compose.yml file

You can check the whole list of Docker commands at https://docs.docker.com/engine/reference/commandline/docker/. And Docker-compose commands at https://docs.docker.com/compose/reference/overview/#command-options-overview-and-help.

Configuring PHPDocker.io

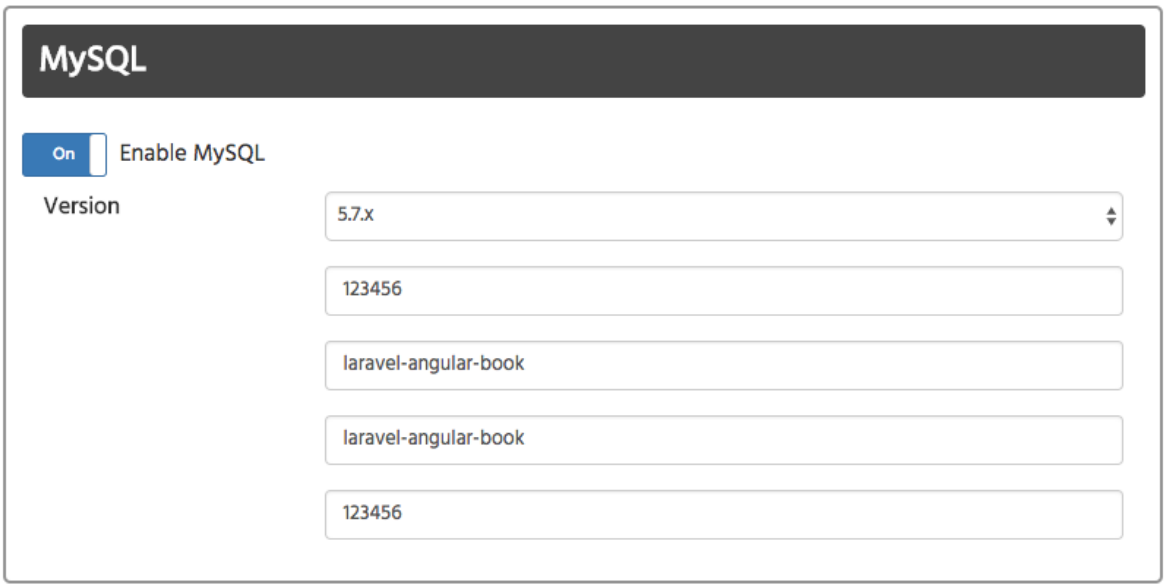
PHPDocker.io is a simple tool that helps us to build PHP applications using the Docker/Container concept with Compose. It's very easy to understand and use; so, let's look at what we need to do:

1. Go to https://phpdocker.io/.
2. Click on the Generator link.
3. Fill out the information, as in the following screenshot.
4. Click on the Generate project archive button and save the folder:



PHPDocker interface

The database configuration is as per the following screenshot:



Database configuration

Note that we are using the latest version of the MYSQL database in the preceding configuration, but you can choose whatever version you prefer. In the following examples, the database version will not matter.

Setting up PHPDocker and Laravel

Now that we have filled in the previous information and downloaded the file for our machine, let's begin setting up our application so as to delve deeper into the directory structure of a Laravel application.

Execute the following steps:

1. Open bash/Terminal/cmd.
2. Go to Users/yourname on Mac and Linux, or C:/ on Windows.
3. Open your Terminal inside the folder and type the following command:

> **composer create-project --prefer-dist laravel/laravel chapter-01**

At the end of your Terminal window, you will see the following result:

> **Writing lock file**
>
> **Generating autoload files**
>
> **> Illuminate\Foundation\ComposerScripts::postUpdate**
>
> **> php artisan optimize**
>
> **Generating optimized class loader**
>
> **php artisan key:generate**

4. In the Terminal window, type:

> **cd chapter-01 && ls**

The results will be as follows:

```
app             composer.json   database        phpunit.xml     resources       tests
artisan         composer.lock   gulpfile.js     public          server.php      vendor
bootstrap       config          package.json    readme.md       storage
```

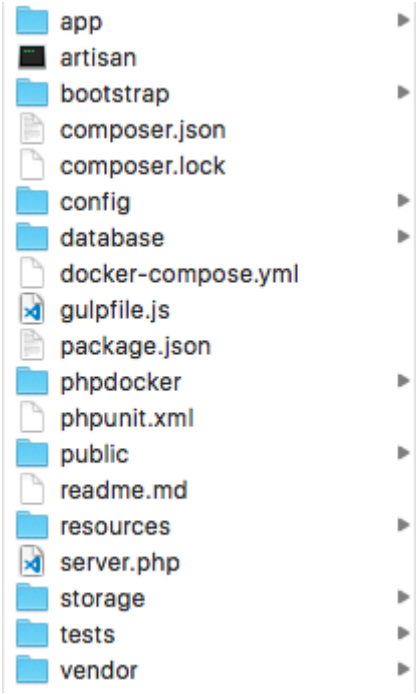
Terminal window output

Congratulations! You have your first Laravel application, built with the Composer package manager.

Now, it's time to join our application with the file downloaded from PHPDocker (our PHP/MySQL Docker screenshot). To do so, follow the next steps.

5. Grab the downloaded archive, hands-on-full-stack-web-development-with-angular-6-and-laravel-5.zip, and unzip it.
6. Copy all of the folder content (a phpdocker folder and a file, docker-compose.yml).
7. Open the chapter-01 folder and paste the content.

Now, inside the chapter-01 folder, we will see the following files:



chapter-01 folder structure

Let's check to make sure that everything will go well with our configuration.

8. Open your Terminal window and type the following command:

```
docker-compose up -d
```

It's important to remember that at this point, you need to have Docker up and running on your machine. If you are completely new to how to run Docker on your machine, you can find more information at https://github.com/docker/labs/tree/master/beginner/.

9. Note that this command may take more time to create and build all of the containers. The results will be as follows:

```
hands-on-full-stack-web-development-with-angular-6-and-laravel-5-memcached ... done
hands-on-full-stack-web-development-with-angular-6-and-laravel-5-webserver ... done
hands-on-full-stack-web-development-with-angular-6-and-laravel-5-mysql     ... done
hands-on-full-stack-web-development-with-angular-6-and-laravel-5-php-fpm   ... done
```

Docker containers up and running

The preceding screenshot indicates that we have started all containers successfully: memcached, webserver (Nginx), mysql, and php-fpm.

Open your browser and type http://localhost:8081; you should see the welcome page for Laravel.

At this point, it is time to open our sample project in a text editor and check all of the Laravel folders and files. You can choose the editor that you are used to, or, if you prefer, you can use the editor that we will describe in the next section.

Installing VS Code text editor

For this chapter, and throughout the book, we will be using **Visual Studio Code** (**VS Code**), a free and highly configurable multiplatform text editor. It is also very useful for working with projects in Angular and TypeScript.

Install VS Code as follows:

1. Go to the download page and choose your platform at https://code.visualstudio.com/Download.
2. Follow the installation steps for your platform.

VS Code has a vibrant community with tons of extensions. You can research and find extensions at https://marketplace.visualstudio.com/VSCode. In the next chapters, we will install and use some of them.

For now, just install VS Code icons from https://marketplace.visualstudio.com/items?itemName=robertohuertasm.vscode-icons.

The basic architecture of Laravel applications

As mentioned previously, Laravel is an MVC framework for the development of modern web applications. It is a software architecture standard that separates the representation of information from users' interaction with it. The architectural standard that it has adopted is not so new; it has been around since the mid-1970s. It remains current, and a number of frameworks still use it today.

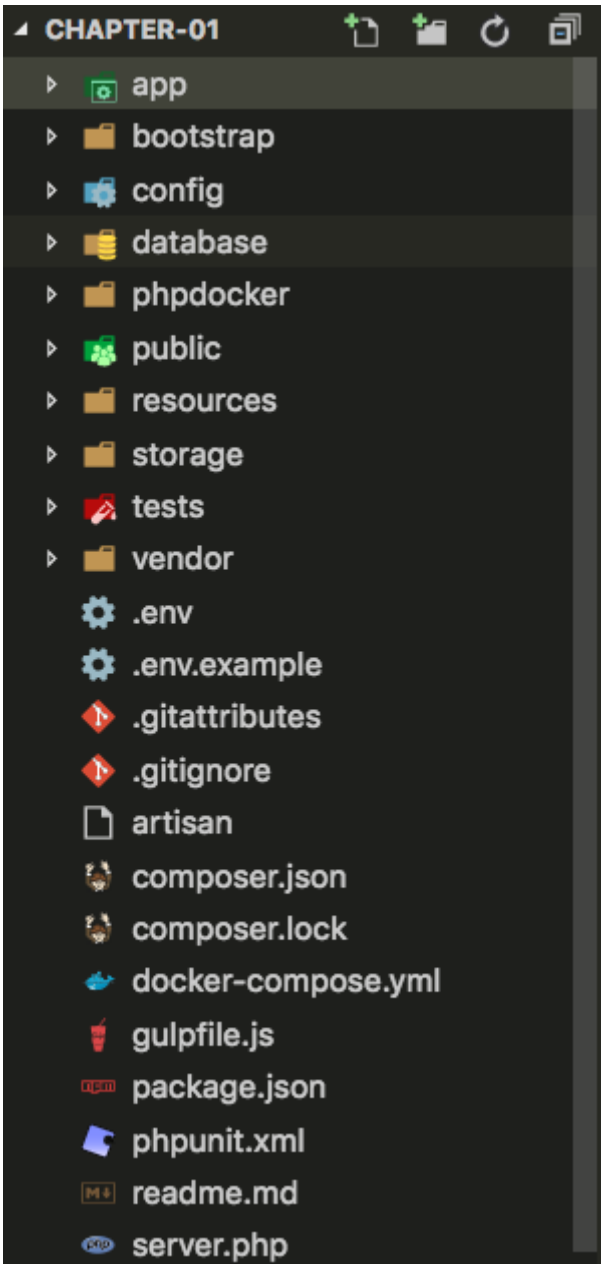
> *You can read more about the MVC pattern at https://en.wikipedia.org/wiki/Model-view-controller.*

Laravel directory structure

Now, let's look at how this pattern is implemented within an application with Laravel:

1. Open the VS Code editor.
2. If this is the first time you are opening VS Code, click on the top menu and navigate to File | Open.
3. Search for the chapter-01 folder, and click Open.
4. Expand the app folder at the left-hand side of VS Code.

The application files are as follows:

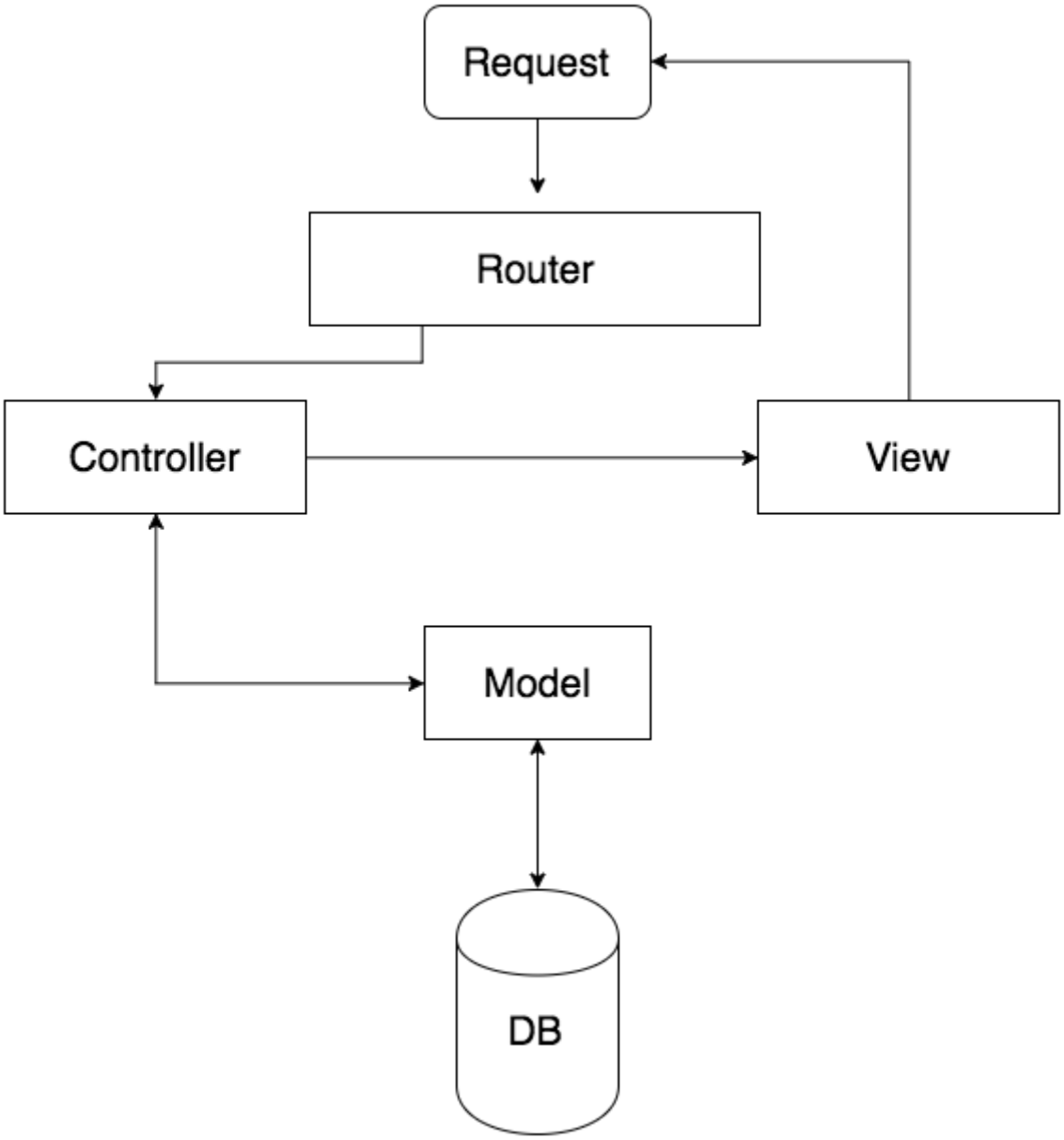


Laravel root folder

The phpdocker folder and docker-compose.yml files are not part of the Laravel framework; we added these files manually, earlier in this chapter.

The MVC flow

In a very basic MVC workflow, when a user interacts with our application, the steps in the following screenshot are performed. Imagine a simple web application about books, with a search input field. When the user types a book name and presses *Enter*, the following flow cycle will occur:



MVC flow

The MVC is represented by the following folders and files:

MVC Architecture	Application Path		File
Model	app/		User.php

View	resources/views		welcome.blade.php
Controller	app/Http/Controllers		Auth/AuthController.php Auth/PasswordController.php

Note that the application models are at the root of the app folder, and the application already has at least one file for MVC implementation.

Also note that the app folder contains all of the core files for our application. The other folders have very intuitive names, such as the following:

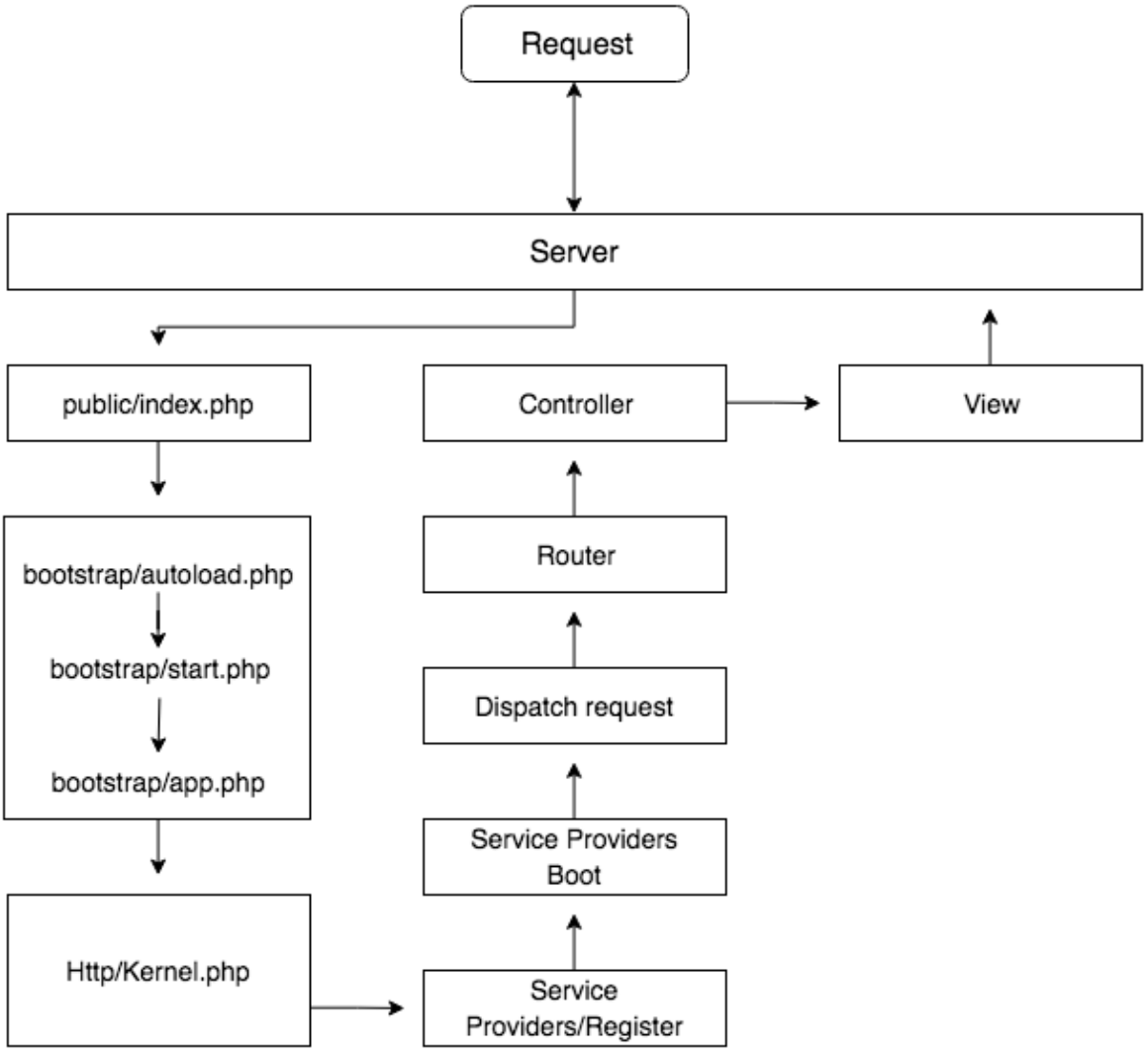
Bootstrap	Cache, autoload, and bootstrap applications
Config	Application's configuration
Database	Factory, migrations, and seeds
Public	JavaScript, CSS, fonts, and images
Resource	Views, SASS/LESS, and localization
Storage	This folder has separated apps, frameworks, and logs
Tests	Unit tests using PHPunit
Vendor	Composer dependencies

Now, let's see how things work in the Laravel structure.

Laravel application life cycle

In a Laravel application, the flow is almost the same as in the previous example, but a little more complex. When the user triggers an event in a browser, the request arrives on a web server (Apache/Nginx), where we have our web application running. So, the server redirects the request into public/index.php, the starting point for the entire framework. In the bootstrap folder, the autoloader.php is started and loads all of the files generated by the composer retrieving an instance to the Laravel application.

Let's look at the following screenshot:



Laravel application cycle

The diagram is complex enough for our first chapter, so we will not get into all of the steps performed by the user's request. Instead, we will go on to another very important feature that is a main concept in Laravel: the Artisan **command-line interface (CLI)**.

You can read more about the request life cycle in Laravel in the official documentation at https://laravel.com/docs/5.2/lifecycle.

Artisan command-line interface

Nowadays, it is common practice to create web applications by using the command line; and, with the evolution of web development tools and technologies, this has become very popular.

We will mention that NPM is one of the most popular. However, for the development of applications using Laravel, we have an advantage. The Artisan CLI is automatically installed when we create a Laravel project.

Let's look at what the official documentation of Laravel says about the Artisan CLI:

Artisan is the name of the command-line interface included with Laravel. It provides a number of helpful commands for your use while developing your application.

Inside of the chapter-01 folder, we find the Artisan bash file. It's responsible for running all of the commands available on the CLI, and there are many of them, to create classes, controllers, seeds, and much more.

After this small introduction to the Artisan CLI, there would be nothing better than looking at some practical examples. So, let's get hands on, and don't forget to start Docker:

1. Open your Terminal window inside the chapter-01 folder, and type the following command:

```
docker-compose up -d
```

2. Let's get inside the php-fpm container and type the following:

```
docker-compose exec php-fpm bash
```

We now have all of the Artisan CLI commands available in the Terminal.

This is the simplest way to interact with the Teminal within our Docker container. If you are using another technique to run the Laravel application, as mentioned at the beginning of the chapter, you do not need to use the following command:

```
docker-compose exec php-fpm bash
```

You can just type the same commands from the next steps into the Terminal.

3. Still in the Terminal, type the following command:

```
php artisan list
```

You will see the framework version and a list of all available commands:

Laravel Framework version 5.2.45

Usage:

command [options] [arguments]

Options:

-h, --help Display this help message

-q, --quiet Do not output any message

-V, --version Display this application version

--ansi Force ANSI output

--no-ansi Disable ANSI output

-n, --no-interaction Do not ask any interactive question

--env[=ENV] The environment the command should run under.

-v|vv|vvv, --verbose Increase the verbosity of messages: 1 for normal output, 2 for more verbose

...

As you can see, the list of commands is very large. Note that the above code snippet, we did not put all the options available with the php artisan list command, but we will see some combinations on next lines.

4. In your Terminal, type the following combination:

php artisan -h migrate

The output will explain exactly what the migrate command can do and what options we have, as seen in the following screenshot:

```
root@26f56807db17:/application# php artisan -h migrate
Usage:
  migrate [options]

Options:
      --database[=DATABASE]  The database connection to use.
      --force                Force the operation to run when in production.
      --path[=PATH]          The path of migrations files to be executed.
      --pretend              Dump the SQL queries that would be run.
      --seed                 Indicates if the seed task should be re-run.
      --step                 Force the migrations to be run so they can be rolled back individually.
  -h, --help                 Display this help message
  -q, --quiet                Do not output any message
  -V, --version              Display this application version
      --ansi                 Force ANSI output
      --no-ansi              Disable ANSI output
  -n, --no-interaction       Do not ask any interactive question
      --env[=ENV]            The environment the command should run under.
  -v|vv|vvv, --verbose       Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and
3 for debug

Help:
 Run the database migrations
root@26f56807db17:/application#
```

Output of php artisan -h migrate

It's also possible to see what options we have for the migrate command.

5. Still in the Terminal, type the following command:

```
php artisan -h make:controller
```

You will see the following output:

```
root@26f56807db17:/application# php artisan -h make:controller
Usage:
  make:controller [options] [--] <name>

Arguments:
  name                  The name of the class

Options:
      --resource        Generate a resource controller class.
  -h, --help            Display this help message
  -q, --quiet           Do not output any message
  -V, --version         Display this application version
      --ansi            Force ANSI output
      --no-ansi         Disable ANSI output
  -n, --no-interaction  Do not ask any interactive question
      --env[=ENV]       The environment the command should run under.
  -v|vv|vvv, --verbose  Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for
 debug

Help:
 Create a new controller class
root@26f56807db17:/application#
```

Output of php artisan -h make:controller

Now, let's look at how to create the MVC in the Laravel application, using the Artisan CLI.

MVC and routes

As mentioned earlier, we will now create a component each of the model, view, and controller, using the Artisan CLI. However, as our heading suggests, we will include another important item: the routes. We have already mentioned them in this chapter (in our diagram of the request life cycle in Laravel, and also in the example diagram of the MVC itself).

In this section, we will focus on creating the file, and checking it after it has been created.

Creating models

Let's get hands on:

1. Open your Terminal window inside the chapter-01 folder, and type the following command:

```
php artisan make:model Band
```

After the command, you should see a success message in green, stating: Model created successfully.

2. Go back to your code editor; inside the app folder, you will see the Band.php file, with the following code:

```
<?php
namespace App;
use Illuminate\Database\Eloquent\Model;
class Band extends Model
{
    //
}
```

Creating controllers

Now it is time to use the artisan to generate our controller, let's see how we can do that:

1. Go back to the Terminal window, and type the following command:

```
php artisan make:controller BandController
```

After the command, you should see a message in green, stating: Controller created successfully.

2. Now, inside app/Http/Controllers, you will see BandController.php, with the following content:

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
class BandController extends Controller
{
   //
}
```

As a good practice, always create your controller with the suffix <Somename>Controller.

Creating views

As we can see earlier when using the `php artisan list` command, we do not have any alias command to create the application views automatically. So we need to create the views manually:

1. Go back to your text editor, and inside the `resources/views` folder, create a new file, named `band.blade.php`.
2. Place the following code inside the `band.blade.php` file:

```
<div class="container">
    <div class="content">
        <div class="title">Hi i'm a view</div>
    </div>
</div>
```

Creating routes

The routes within Laravel are responsible for directing all HTTP traffic coming from the user's requests, so the routes are responsible for the entire inflow in a Laravel application, as we saw in the preceding diagrams.

In this section, we will briefly look at the types of routes available in Laravel, and how to create a simple route for our MVC component.

At this point, it is only necessary to look at how the routes work. Later in the book, we will get deeper into application routing.

So, let's look at what we can use to handle routes in Laravel:

Code	HTTP | METHOD |Verb
Route::get($uri, $callback);	GET
Route::post($uri, $callback);	POST
Route::put($uri, $callback);	PUT
Route::patch($uri, $callback);	PATCH
Route::delete($uri, $callback);	DELETE
Route::options($uri, $callback);	OPTIONS

Each of the routes available is responsible for handling one type of HTTP request method. Also, we can combine more than one method in the same route, as in the following code. Do not be too concerned with this now; we'll see how to deal with this type of routing later in the book:

```
Route::match(['get', 'post'], '/', function () {
    //
});
```

Now, let's create our first route:

1. On your text editor, open web.php inside the routes folder, and add the following code, right after the welcome view:

```
Route::get('/band', function () {
return view('band');
});
```

2. Open your browser to http://localhost:8081/band, and you will see the following message:

```
Hi i'm a view
```

Don't forget to start all Docker containers using the docker-compose up -d command. If you followed the previous examples, you will already have everything up and running.

Bravo! We have created our first route. It is a simple example, but we have all of the things in place and working well. In the next section, we'll look at how to integrate a model with a controller and render the view.

Connecting with a database

As we saw previously, the controllers are activated by the routes and transmit information between the model /database and the view. In the preceding example, we used static content inside the view, but in larger applications, we will almost always have content coming from a database, or generated within the controller and passed to the view.

In the next example, we will see how to do this.

Setting up the database inside a Docker container

It's now time to configure our database. If you use Homestead, you probably have your database connection configured and working well. To check, open your Terminal and type the following command:

> **php artisan tinker**
>
> **DB::connection()->getPdo();**

If everything goes well, you will see the following message:

```
Psy Shell v0.7.2 (PHP 7.2.2-3+ubuntu16.04.1+deb.sury.org+1 — cli) by Justin Hileman
>>> DB::connection()->getPdo();
=> PDO {#639
     inTransaction: false,
     attributes: {
       CASE: NATURAL,
       ERRMODE: EXCEPTION,
       AUTOCOMMIT: 1,
       PERSISTENT: false,
       DRIVER_NAME: "mysql",
       SERVER_INFO: "Uptime: 27  Threads: 1  Questions: 9  Slow queries: 0  Opens: 105  Flush tables: 1  Open tables: 98  Queries
 per second avg: 0.333",
       ORACLE_NULLS: NATURAL,
       CLIENT_VERSION: "mysqlnd 5.0.12-dev - 20150407 - $Id: 38fea24f2847fa7519001be390c98ae0acafe387 $",
       SERVER_VERSION: "5.7.21",
       STATEMENT_CLASS: [
         "PDOStatement",
       ],
       EMULATE_PREPARES: 0,
       CONNECTION_STATUS: "mysql via TCP/IP",
       DEFAULT_FETCH_MODE: BOTH,
     },
   }
>>>
```

Database connection message

For this example, however, we are using Docker, and we need to do some configuration to accomplish this task:

1. Inside of the root project, open the .env file and look at line 8 (the database connection), which looks as follows:

> DB_CONNECTION=mysql
>
> DB_HOST=127.0.0.1
>
> DB_PORT=3306
>
> DB_DATABASE=homestead
>
> DB_USERNAME=homestead
>
> DB_PASSWORD=secret

Now, replace the preceding code with the following lines:

> DB_CONNECTION=mysql
>
> DB_HOST=mysql

```
DB_PORT=3306
DB_DATABASE=laravel-angular-book
DB_USERNAME=laravel-angular-book
DB_PASSWORD=123456
```

Note that we need to change a bit to get the Docker MySQL container directions; if you don't remember what you chose in the PHPDocker.io generator, you can copy it from the container configuration.

2. Open docker-compose.yml at the root directory.
3. Copy the environment variables from the MySQL container setup:

```
mysql:
  image: mysql:8.0
  entrypoint: ['/entrypoint.sh', '--character-set-server=utf8', '--
  collation-server=utf8_general_ci']
  container_name: larahell-mysql
  working_dir: /application
  volumes:
    - .:/application
  environment:
    - MYSQL_ROOT_PASSWORD=larahell
    - MYSQL_DATABASE=larahell-angular-book
    - MYSQL_USER=larahell-user
    - MYSQL_PASSWORD=123456
  ports:
    - "8083:3306"
```

Now, it's time to test our connection.

4. In your Terminal window, type the following command:

```
docker-compose exec php-fpm bash
```

5. Finally, let's check our connection; type the following command:

```
php artisan tinker
DB::connection()->getPdo();
```

You should see the same message as the previous screenshot. Then, you will have everything you need to go ahead with the example.

Creating a migrations file and database seed

Migration files are very common in some MVC frameworks, such as Rails, Django, and, of course, Laravel. It is through this type of file that we can keep our database consistent with our application, since we cannot versioning the database schemes . Migration files help us to store each change in our database, so that we can version these files and keep the project consistent.

Database seeds serve to populate the tables of a database with an initial batch of records; this is extremely useful when we are developing web applications from the beginning. The data of the initial load can be varied, from tables of users to administration objects such as passwords and tokens, and everything else that we require.

Let's look at how we can create a migration file for the Bands model in Laravel:

1. Open your Terminal window and type the following command:

```
php artisan make:migration create_bands_table
```

2. Open the database/migrations folder, and you will see a file called<timestamp>create_bands_table.php.
3. Open this file and paste the following code inside public function up():

```
Schema::create('bands', function (Blueprint $table) {
  $table->increments('id');
  $table->string('name');
  $table->string('description');
  $table->timestamps();
});
```

4. Paste the following code inside public function down():

```
Schema::dropIfExists('bands');
```

5. The final result will be the following code:

```
<?php
use Illuminate\Support\Facades\Schema;
 use Illuminate\Database\Schema\Blueprint;
 use Illuminate\Database\Migrations\Migration;
```

```
class CreateBandsTable extends Migration
{
    /**
    * Run the migrations.
    *
    * @return void
   */
    public function up()
    {
        Schema::create('bands', function (Blueprint $table) {
        $table->increments('id');
        $table->string('name');
        $table->string('description');
        $table->timestamps();
        });
    }
   /**
    * Reverse the migrations.
    *
    * @return void
    */
    public function down()
    {
        Schema::dropIfExists('bands');
    }
}
```

6. Inside of the database/factories folder, open the ModalFactory.php file and add the following code, right after the User Factory. Note that we are using a PHP library called faker inside a factory function, in order to generate some data:

```
$factory->define(App\Band::class, function (Faker\Generator $faker) {
return [
 'name' => $faker->word,
 'description' => $faker->sentence
 ];
 });
```

7. Go back to your Terminal window and create a database seed. To do this, type the following command:

```
php artisan make:seeder BandsTableSeeder
```

8. In the database/seeds folder, open the BandsTableSeeder.php file and type the following code, inside public function run():

```
factory(App\Band::class,5)->create()->each(function ($p) {
$p->save();
});
```

9. Now, in the database/seeds folder, open the DatabaseSeeder.php file and add the following code, inside public function run():

```
$this->call(BandsTableSeeder::class);
```

You can read more about Faker PHP at https://github.com/fzaninotto/Faker.

Before we go any further , we need to do a small refactoring on the Band model.

10. Inside of the app root, open the Band.php file and add the following code, inside the Band class:

```
protected $fillable = ['name','description'];
```

11. Go back to your Terminal and type the following command:

```
php artisan migrate
```

After the command, you will see the following message in the Terminal window:

```
Migration table created successfully.
```

The preceding command was just to populate the database with our seed.

12. Go back to your Terminal and type the following command:

```
php artisan db:seed
```

We now have five items ready to use in our database.

Let's check whether everything will go smoothly.

13. Inside of your Terminal, to exit php-fpm container, type the following command:

```
exit
```

14. Now, in the application root folder, type the following command in your Terminal:

```
docker-compose exec mysql mysql -ularavel-angular-book -p123456
```

The preceding command will give you access to the MySQL console inside mysql Docker container, almost exactly the same as how we gained access to php-fpm container.

15. Inside of the Terminal, type the following command to see all of the databases:

```
show databases;
```

As you can see, we have two tables: information_schema and laravel-angular-book.

16. Let's access the laravel-angular-book table; type the following command:

```
use laravel-angular-book;
```

17. And now, let's check our tables, as follows:

```
show tables;
```

18. Now, let's SELECT all records from the bands tables:

```
SELECT * from bands;
```

We will see something similar to the following screenshot:

```
mysql> SELECT * from bands;
+----+--------+-------------------------------------------------------------+
| id | name   | description                                                 |
+----+--------+-------------------------------------------------------------+
|  1 | quidem | Sed sed rerum autem accusamus assumenda quia exercitationem. |
|  2 | at     | Deleniti eos quas eum consectetur.                          |
|  3 | totam  | Voluptates quibusdam non vel quia sed et et.                |
|  4 | alias  | Adipisci mollitia ipsum iste harum maiores.                 |
|  5 | libero | Quia tempore quia numquam ad.                               |
+----+--------+-------------------------------------------------------------+
5 rows in set (0.00 sec)
```

Database bands table

19. Now, exit the MySQL console with the following command:

```
exit
```

Using the resource flag to create CRUD methods

Let's see another feature of the Artisan CLI, creating all of the **Create**, **Read**, **Update**, **and Delete** (**CRUD**) operations using a single command.

First, in the app/Http/Controllers folder, delete the BandController.php file:

1. Open your Terminal window and type the following command:

> **php artisan make:controller BandController --resource**

This action will create the same file again, but now, it includes the CRUD operations, as shown in the following code:

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class BandController extends Controller
 {
    /**
    * Display a listing of the resource.
    *
    * @return \Illuminate\Http\Response
    */
    public function index()
    {
        //
    }
   /**
    * Show the form for creating a new resource.
    *
    * @return \Illuminate\Http\Response
    */
    public function create()
    {
```

```
        //
    }
    /**
     * Store a newly created resource in storage.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        //
    }
    /**
     * Display the specified resource.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function show($id)
    {
        //
    }
    /**
     * Show the form for editing the specified resource.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function edit($id)
    {
        //
    }
    /**
     * Update the specified resource in storage.
     *
     * @param \Illuminate\Http\Request $request
```

```
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function update(Request $request, $id)
    {
        //
    }
    /**
     * Remove the specified resource from storage.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function destroy($id)
    {
        //
    }
}
```

For this example, we will write only two methods: one to list all of the records, and another to get a specific record. Don't worry about the other methods; we will cover all of the methods in the upcoming chapters.

2. Let's edit public function index() and add the following code:

```
$bands = Band::all();
 return $bands;
```

3. Now, edit public function show() and add the following code:

```
$band = Band::find($id);
 return view('bands.show', array('band' => $band));
```

4. Add the following line, right after App\Http\Requests:

```
use App\Band;
```

5. Update the routes.php file, inside the routes folder, to the following code:

```
Route::get('/', function () {
 return view('welcome');
```

```
 });
Route::resource('bands', 'BandController');
```

6. Open your browser and go to http://localhost:8081/bands, where you will see the following content:

```
[{
  "id": 1,
  "name": "porro",
  "description": "Minus sapiente ut libero explicabo et voluptas harum.",
  "created_at": "2018-03-02 19:20:58",
  "updated_at": "2018-03-02 19:20:58"}
...]
```

Don't worry if your data is different from the previous code; this is due to Faker generating random data. Note that we are returning a JSON directly to the browser, instead of returning the data to the view. This is a very important feature of Laravel; it serializes and deserializes data, by default.

Creating the Blade template engine

Now, it's time to create another view component. This time, we will use the Blade template engine to show some records from our database. Let's look at what the official documentation says about Blade:

Blade is the simple, yet powerful, templating engine provided with Laravel. Unlike other popular PHP templating engines, Blade does not restrict you from using plain PHP code in your views. All Blade views are compiled into plain PHP code and cached until they are modified, meaning Blade adds essentially zero overhead to your application.

Now, it's time to see this behavior in action:

1. Go back to the code editor and create another folder inside resources/views, called bands.
2. Create a file, show.blade.php, inside resources/views/bands, and place the following code in it:

```
<h1>Band {{ $band->id }}</h1>
<ul>
<li>band: {{ $band->name }}</li>
<li>description: {{ $band->description }}</li>
</ul>
```

You can find out more about Blade at https://laravel.com/docs/5.2/blade.

3. Open your browser to http://localhost:8081/bands/1. You will see the template in action, with results similar to the following:

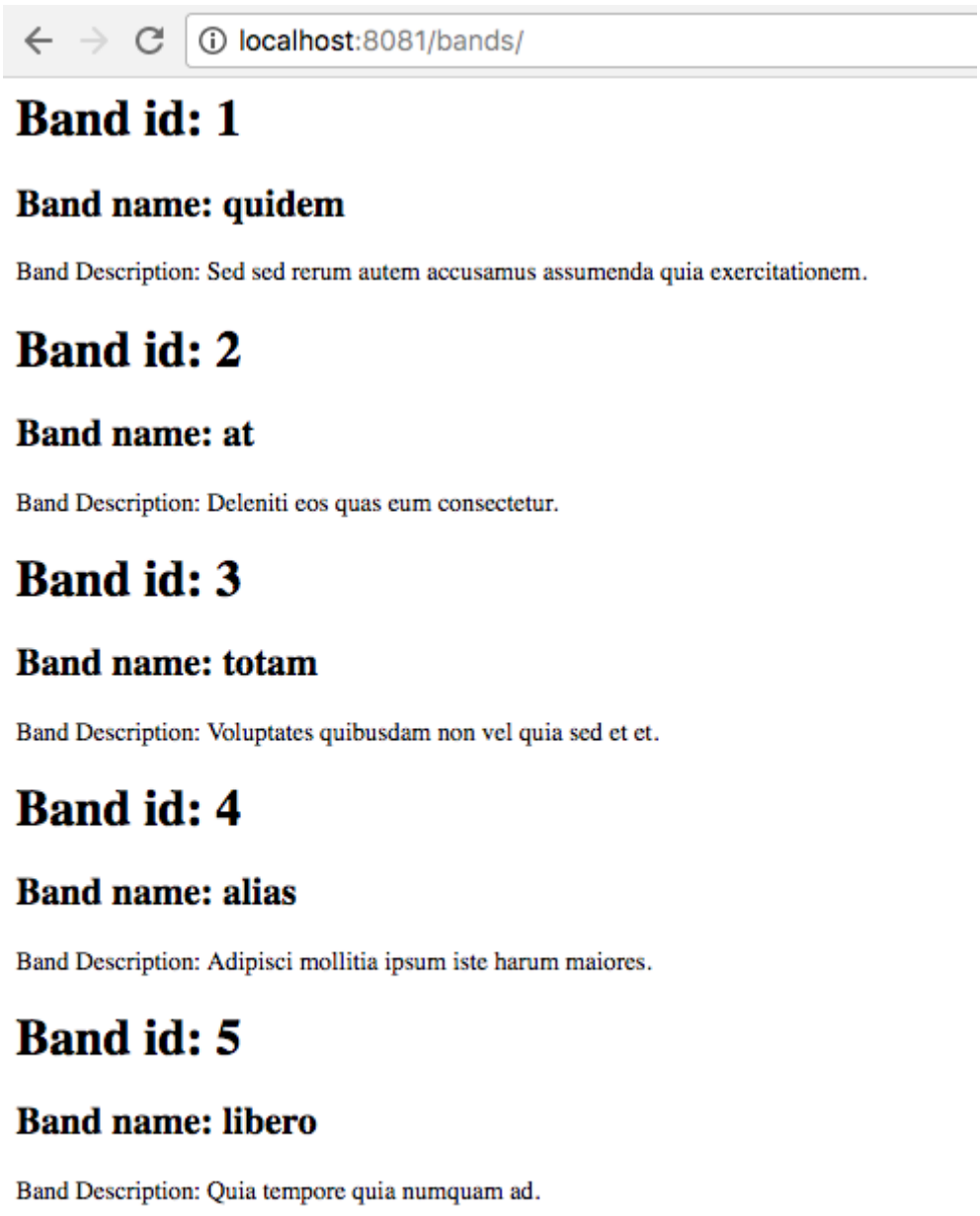
View of the template engine

Note that here, we are using the Blade template engine to show a record from our database. Now, let's create another view to render all of the records.

4. Create another file, called index.blade.php, inside resources/views/bands, and place the following code in it:

```
@foreach ($bands as $band)
<h1>Band id: {{ $band->id }}</h1>
<h2>Band name: {{ $band->name }}</h2>
<p>Band Description: {{ $band->description }}</p>
@endforeach
```

5. Go back to your browser and visit http://localhost:8081/bands/, where you will see a result similar to the following:



View template engine

Summary

We have finally finished the first chapter, and we have covered many of the core concepts of the Laravel framework. Even with the simple examples that we discussed in this chapter, we have provided a relevant basis for all of Laravel's functionality. It would be possible to create incredible applications with only this knowledge. However, we intend to delve deeper into some concepts that deserve separate chapters. Throughout the book, we will create an entire application, using a RESTful API, Angular, and some other tools, such as TypeScript, which we will look at in the next chapter.